

Memoria Práctica 4

Aprendizaje Automático

Jorge Alcalde Vesteiro
Andy Barcia Rodríguez
Pedro Pablo López Mena

Índice

1) Introducción	3
1.a) ¿Qué es una SVM y como funciona?	3
1.b) Organización de la práctica	3
2) Práctica Guiada	3
2.a) Problema linealmente separable	7
2.a.a) Comparación C	8
2.b) Problema quasi linealmente separable.	8
2.c) Problema no separable linealmente 1	9
2.c.a) Kernel lineal	10
2.c.b) Kernel Polinómico	10
2.c.c) Comparación	11
2.c.d) ¿Como funciona el kernel polinómico?	11
2.d) Problema no separable linealmente 2	11
2.d.a) Kernel lineal	12
2.d.b) Kernel Polinómico	12
2.d.c) Kernel Radial	13
2.d.d) Comparación	14
2.d.e) ¿Como funciona el kernel radial?	14
3) Caso Práctico	14
3.a) Preprocessing	14
3.b) Baseline	15
3.c) Reducción de dimensionalidad con PCA	16
3.d) Búsqueda de parámetros	16

1) Introducción

Para esta práctica vamos a realizar un estudio y una comprensión sobre las SVM (Support Vector Machines).

Para ello vamos a:

- Profundizar en los parámetros que comprenden a las SVM (kernels, hiperplanos, márgenes, etc..)
- Analizar C respecto a los resultados
- Conocer los kernels más importantes

1.a) ¿Qué es una SVM y como funciona?

Un SVM es una de las técnicas de aprendizaje no supervisado más potentes que se encuentran en el aprendizaje automático.

A grandes rasgos un SVM busca definir un hiperplano que sea capaz de separar las clases que se le presentan al modelo (punto, línea, superficie, etc dependiendo de las dimensiones). Una de sus características más interesantes es que dentro del aprendizaje, los puntos que más importancia tienen son los Support Vectors, que se corresponden con los puntos que están en los márgenes del plano.

Para esta práctica vamos a jugar con dos de las características más importantes de los SVMs.

- La C es el parámetro que controla dentro de la ecuación la importancia que se le da al hecho de que el modelo tenga outliers fuera de sus márgenes. Si la C es pequeña, el modelo es permisivo, por lo que los márgenes serán más grandes, por el contrario, cuando aumentas la C se reduce la permisividad, siendo una $C = \infty$ la que denominaría márgenes duros (0 permisividad con los outliers)
- Por otro lado los kernels son funciones que se integran dentro de las SVMs para aplicar transformaciones sobre los puntos. Imaginemos unos puntos que solo se pueden separar por una circunferencia. Como no se pueden separar por una función de línea «típica» lo que podemos hacer es transformar los puntos siguiendo la ecuación de la circunferencia $X^2 + Y^2$. Ahora en este plano transformado los puntos SI que se pueden separar por una línea y unos márgenes. Una vez separados solo quedaría «destransformar» la ecuación, por lo que la recta se transforma para nosotros en una circunferencia.
- Por último, gamma es el parámetro que se usa en sklearn para denotar la importancia por cada ejemplo de entrenamiento (un poco como el learning rate). De esta manera si se usan gammas altas se puede encontrar overfitting y con gammas bajas se puede no converger (o tardar mucho).

1.b) Organización de la práctica

Esta práctica se divide en dos apartados.

- El primer apartado es un estudio guiado sobre *datasets* sintéticos de tal manera que podamos observar las características principales de los SVM
- El segundo apartado es un caso práctico donde se evaluará un *dataset* de canciones de 13 características y 2 clases. Para este apartado hay que intentar conseguir el mejor rendimiento posible en un 5CV teniendo la libertad de usar los parámetros que uno quiera

2) Práctica Guiada

En este apartado vamos a utilizar *datasets* sintéticos creados para ver como funcionan los kernels claramente

Para estos apartados vamos a usar las siguientes funciones:

Para realizar las pruebas vamos a utilizar la validación cruzada y como decisión para escoger los modelos vamos a usar el acierto en la validación

```
def generic_svm_cv(
    dataset,
    param_dicts,
    x_columns = ["x1", "x2"],
    y_column = "y",
    in_split = 4,
    out_split = 3,
    metric = "f1",
    verbose = 2,
    n_jobs = 4,
):
    X = dataset[x_columns].values
    Y = dataset[y_column].values

    fuera_cv = StratifiedKFold(n_splits=out_split, shuffle=True, random_state=42)
    dentro_cv = StratifiedKFold(n_splits=in_split, shuffle=True, random_state=42)

    results_fuera = {
        "best_index": []
    }

    for i, (ixs_train_fuera, ixs_test_fuera) in enumerate(fuera_cv.split(X, Y)):
        # Datos de entrenamiento para el bucle exterior
        X_train_dentro = X[ixs_train_fuera]
        Y_train_dentro = Y[ixs_train_fuera]

        multiple_fits = GridSearchCV(
            SVC(),
            param_dicts,
            cv = dentro_cv,
            scoring = metric,
            verbose=verbose,
            n_jobs=n_jobs,
        )

        multiple_fits.fit(X_train_dentro, Y_train_dentro)

        results_fuera["best_index"].append(multiple_fits.best_index_)
        results_fuera["params"] = multiple_fits.cv_results_["params"]

    results = {}

    best_index = st.mode(results_fuera["best_index"])
    best_model = SVC(**results_fuera["params"][best_index])

    cross_validation_fuera = cross_validate(
        best_model,
        X, Y,
        cv = fuera_cv,
        return_estimator = True,
        scoring = metric,
    )
```

```

        results = {
            'best': cross_validation_fuera["estimator"]
[ np.argmax(cross_validation_fuera["test_score"]) ],
            'params': results_fuera["params"][best_index],
            'scores': cross_validation_fuera["test_score"]
        }

    return results

```

```

def generic_simple_cv(
    dataset,
    param_dicts,
    x_columns = ["x1", "x2"],
    y_column = "y",
    out_split = 5,
    metric = "f1",
    verbose = 2,
    n_jobs = 4,
):
    X = dataset[x_columns].values
    Y = dataset[y_column].values

    fuera_cv = StratifiedKFold(n_splits=out_split, shuffle=True, random_state=42)

    cross_validation_fuera = GridSearchCV(
        SVC(),
        param_dicts,
        cv = fuera_cv,
        scoring = metric,
        verbose=verbose,
        n_jobs=n_jobs,
    )

    cross_validation_fuera.fit(X, Y)

    best_index = cross_validation_fuera.best_index_

    results = {
        'best': cross_validation_fuera.best_estimator_,
        'params': cross_validation_fuera.cv_results_["params"][best_index],
        'scores': cross_validation_fuera.cv_results_["mean_test_score"],
        'best_score': cross_validation_fuera.best_score_,
    }

    return results

```

Por otro lado para dibujar distintas gráficas con los datos del dataset y las predicciones del SVM (junto con sus vectores de soporte).

```

def separate_by_class(dataset):
    is_class_0 = dataset["y"] == 0
    is_class_1 = ~is_class_0

```

```

d1 = dataset[is_class_0].reset_index(drop=True)
d2 = dataset[is_class_1].reset_index(drop=True)
return d1, d2

def scatter_plot(X, plotline=False, offset=0.3, nombre = None):
    clase1, clase2 = separate_by_class(X)

    plt.scatter(clase1["x1"], clase1["x2"], label = "0")
    plt.scatter(clase2["x1"], clase2["x2"], label = "1")

    if plotline:
        x_min = min(np.min(clase1["x1"]), np.min(clase2["x2"]))
        y_min = min(np.min(clase1["x2"]), np.min(clase2["x2"]))

        rel_height = lambda x, y: (y-y_min)/(x-x_min)

        c1_min = np.argmin([rel_height(x,y) for x,y in zip(clase1["x1"], clase1["x2"])]))
        c2_max = np.argmax([rel_height(x,y) for x,y in zip(clase2["x1"], clase2["x2"])]))

        p1 = [clase1["x1"][c1_min], clase1["x2"][c1_min]-offset]
        p2 = [clase2["x1"][c2_max], clase2["x2"][c2_max]+offset]

        plt.scatter(p1[0], p1[1], marker="x")
        plt.scatter(p2[0], p2[1], marker="x")

        plt.axline(p1, p2)

    plt.legend()
    plt.savefig(f"Scatter_{nombre}.png", bbox_inches='tight')
    plt.plot()

def plot_boundaries_and_support_vectors(X, estimator, draw_support_vectors=True,
size=50, nombre = None):
    ax = plt.gca()
    DecisionBoundaryDisplay.from_estimator(
        estimator,
        X[["x1", "x2"]].values,
        plot_method='contourf',
        colors=["blue", "orange"],
        levels=[-1, 0, 1],
        alpha=0.4,
        linestyle=["--", "-", "--"],
        ax=ax,
    )

    clase1, clase2 = separate_by_class(X)
    plt.scatter(clase1["x1"], clase1["x2"], label = "0", color="blue", s=size)
    plt.scatter(clase2["x1"], clase2["x2"], label = "1", color="orange", s=size)

    if draw_support_vectors:
        support_vectors = estimator.support_vectors_
        plt.scatter(
            support_vectors[:,0],
            support_vectors[:,1],

```

```

        s=int(size*1.10),
        linewidth=1,
        facecolors="none",
        edgecolors="k",
    )

plt.title("Boundaries")

plt.xlabel('x')
plt.ylabel('y')

plt.legend()

plt.savefig(f"SVM_{nombre}.png", bbox_inches='tight')
plt.plot()

```

2.a) Problema linealmente separable

En este caso nos encontramos con un problema que es perfectamente separable en 2 clases.

Esto se puede observar visualmente, ya que podemos trazar una linea a mano (en este caso usando la opción `plotline` definida anteriormente en las funciones que coge el punto más parecido a la clase contraria como `edges`, y aumenta un *threshold* para que ni siquiera esos estén definidos)

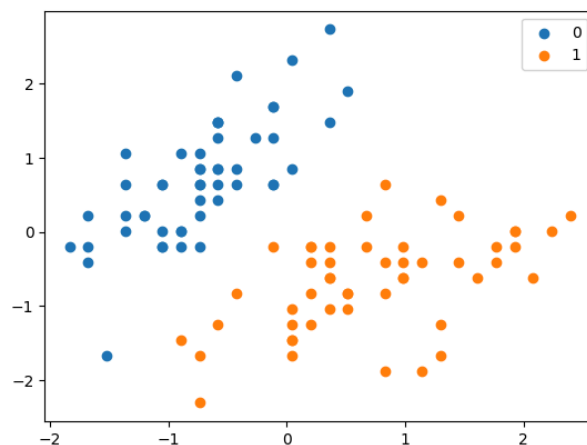


Figura 1: Scatter de puntos del apartado 1

Aquí podemos ver una comparación de la linea realizada a mano frente a la linea calculada con el SVC con $C = 100$.

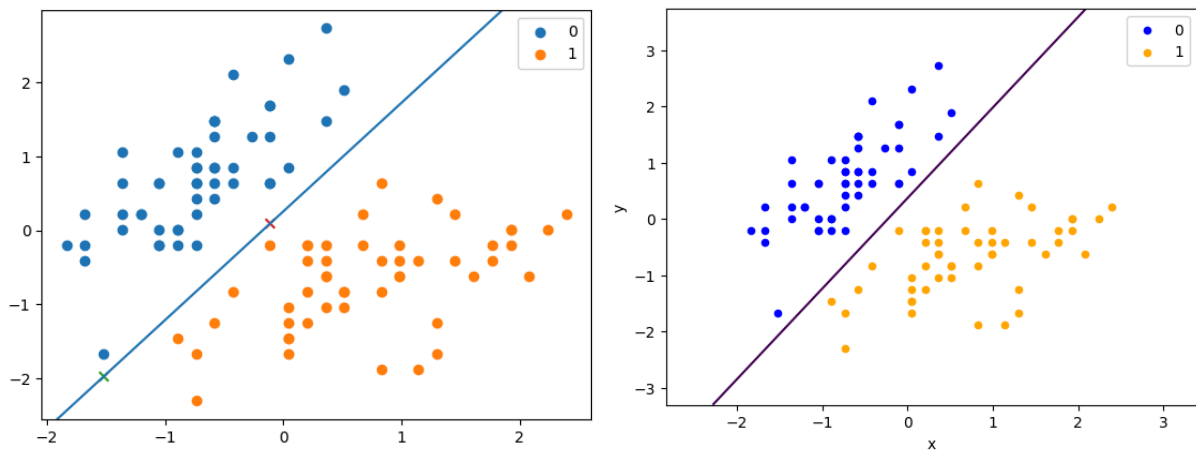


Figura 2: Línea a mano vs línea de svm

2.a.a) Comparación C

Ahora vamos a entrenar dos SVM con $C=100$ y $C=1$ para visualizar como realizan sus operaciones

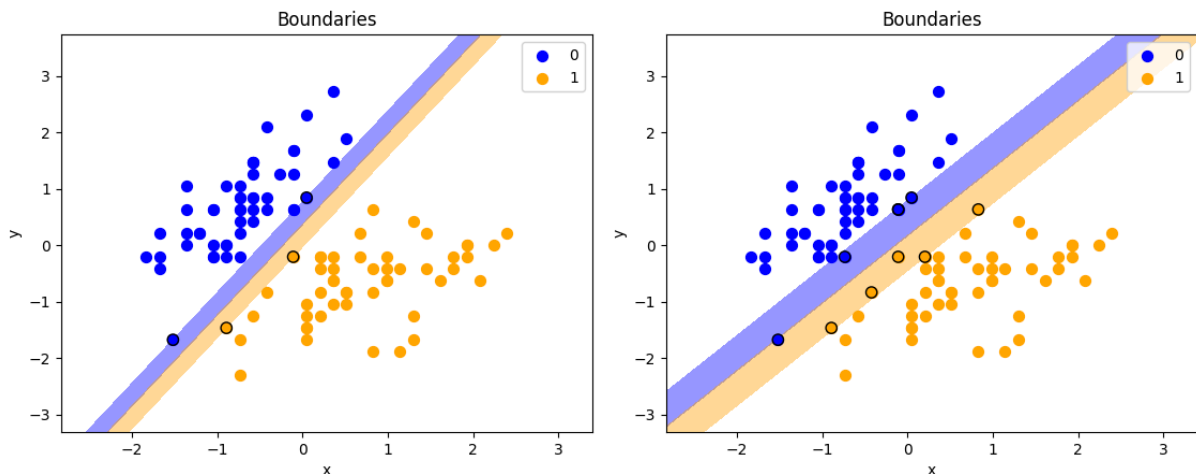


Figura 3: SVM con $C=100$ (izquierda) y $C=1$ (derecha).

A través de esta imagen podemos corroborar lo que se ha mencionado anteriormente:

- Según la teoría, cuanto más grande sea C , más restrictivo y menos tolerante se es con los outliers, por lo que por definición los márgenes serán más finos
- Por otro lado, una C más pequeña es más permisiva, por lo que se puede observar que hay varios puntos que se sitúan dentro de los márgenes
- En este caso ambas divisiones producen un 100% de accuracy separando los elementos, ya que los datos sintéticos son muy sencillos, sin embargo, podemos intuir en un futuro las consecuencias que tiene ser más o menos restrictivo.

2.b) Problema quasi linealmente separable.

Los datos de este apartado son bastante similares a los del apartado anterior; la diferencia es que, si bien parece que se podrían dividir de manera lineal, en este caso no se puede, porque hay algunos puntos que se colarían dentro de las regiones del otro.

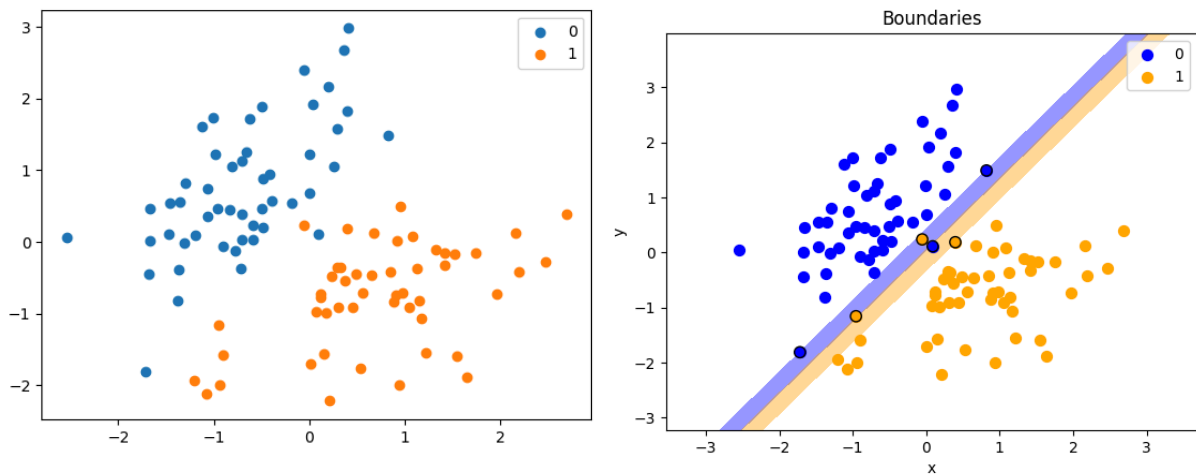


Figura 4: SVM 1.2

```
params = {
    "C": (100, 1, 5, 10, 25, 50, 250),
    "kernel": ("linear",)
}
resultados_p2 = generic_svm_cv(dataset2, params)
resultados_p2
```

```
{'best': SVC(C=100, kernel='linear'),
 'params': {'C': 100, 'kernel': 'linear'},
 'scores': array([1.          , 1.          , 0.94444444]),
 'mean_score': 0.9814814814814815}
```

En este caso se realiza simplemente una exploración, a través de la validación cruzada hemos obtenido un modelo bastante preciso.

Nótese que el modelo está entrenado con validación cruzada anidada, por lo que se puede ver en los *scores* obtenidos que el *accuracy* puede llegar a ser de 1 (datos linealmente separables) dependiendo de la suerte que tengan los distintos *folds* al repartirse los datos que se encuentran cerca del margen.

2.c) Problema no separable linealmente 1

En este apartado podemos ver al dibujar las líneas de puntos que no hay manera de que estos valores sean separables linealmente, puesto que tienen forma de espiral.

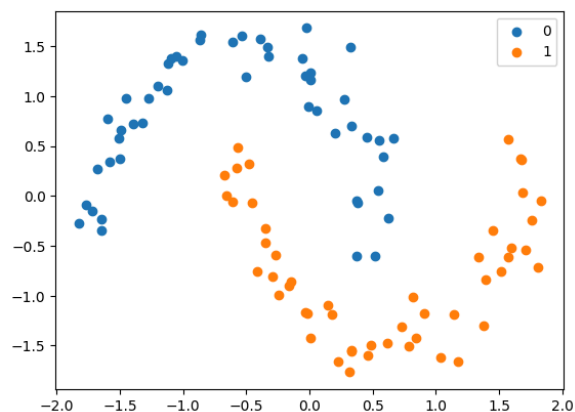


Figura 5: Scatter de puntos del apartado 3

2.c.a) Kernel lineal

Para realizar las pruebas lo primero que se nos pide es usar un kernel lineal. Visualmente podemos ver que no hay manera en que este problema sea separable linealmente por lo que ya partimos de que que es imposible sacar un 100%.

Sin embargo, usando un kernel lineal y una $C = 100$ (a través de un CV anidado) podemos ver que obtenemos un f1 promedio del 84%, que no está mal.

```
params = {
    "C": (100, 1, 5, 10, 25, 50, 250),
    "kernel": ("linear",)
}
resultados_p3 = generic_svm_cv(dataset3, params)
resultados_p3

{'best': SVC(C=100, kernel='linear'),
 'params': {'C': 100, 'kernel': 'linear'},
 'scores': array([0.89473684, 0.83870968, 0.78571429]),
 'mean_score': 0.8397202684129679}
```

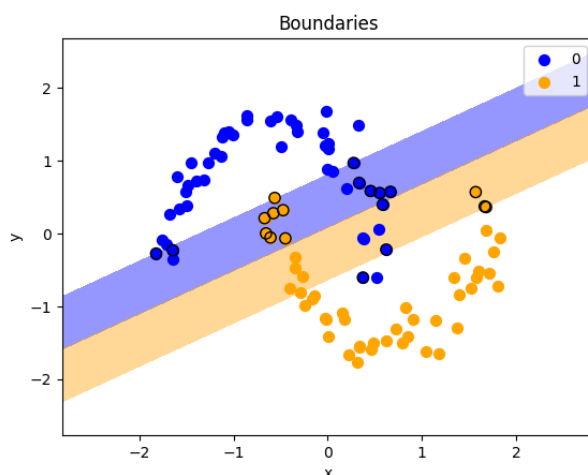


Figura 6: SVM con kernel lineal para datos no linealmente separables.

2.c.b) Kernel Polinómico

Ahora, teniendo la oportunidad de usar otros kernels, vamos a usar un kernel polinómico de grado 3 (que tiene una función que se podría asemejar bastante a la distribución de puntos).

Realizando la validación cruzada observamos que la precisión encontrada es del 98%, superando por un razonable 15% a lo que conseguíamos con el kernel lineal.

```
params = {
    "C": (100, 1, 5, 10, 25, 50, 250),
    "kernel": ("poly",),
    "degree": (3,),
    "coef0": (1.0, 2.0, 5.0, 15.0, 30.0),
}
resultados_p3 = generic_svm_cv(dataset3, params)
resultados_p3
```

```
{'best': SVC(C=1, coef0=1.0, kernel='poly'),
'params': {'C': 1, 'coef0': 1.0, 'degree': 3, 'kernel': 'poly'},
'scores': array([0.97142857, 1.          , 0.96969697]),
'mean_score': 0.9803751803751803}
```

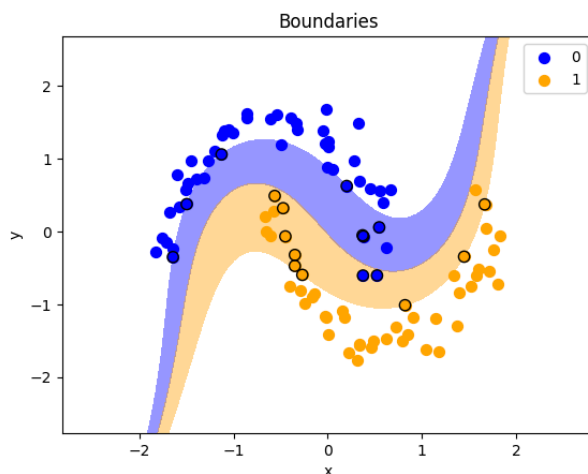


Figura 7: SVM 1.3

2.c.c) Comparación

Podemos ver lo siguiente:

- Por un lado, el kernel polinómico es mejor que el lineal, lo cual se podría intuir desde un principio viendo la forma de espiral que presentaban los puntos pero que hemos confirmado empíricamente.
- La diferencia que es del 15% en papel no es desmesurada principalmente porque el kernel lineal es mejor de lo que se pueda pensar. Sin embargo podemos ver como un kernel que tenga una forma más parecida al problema nos da la posibilidad de encontrar soluciones mejores (98% f1 en este caso)

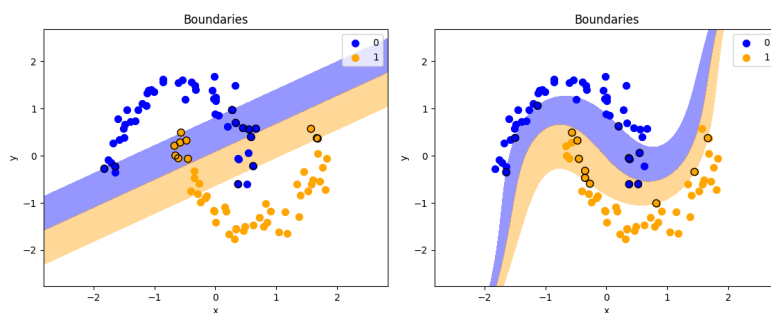


Figura 8: SVM 1.3

2.c.d) ¿Como funciona el kernel polinómico?

El kernel polinómico es una de las soluciones que se presentan para hacer transformaciones no lineales. En este caso, el kernel polinómico sigue la estructura de un polinomio de grado N: $(\alpha * \langle x, y \rangle + \beta)^N$

2.d) Problema no separable linealmente 2

La última distribución de puntos sintéticos que encontramos es en forma de circunferencias concéntricas. De nuevo, podemos ver que no hay manera que usando un kernel lineal separemos los puntos en dos clases así que vamos a requerir de transformaciones.

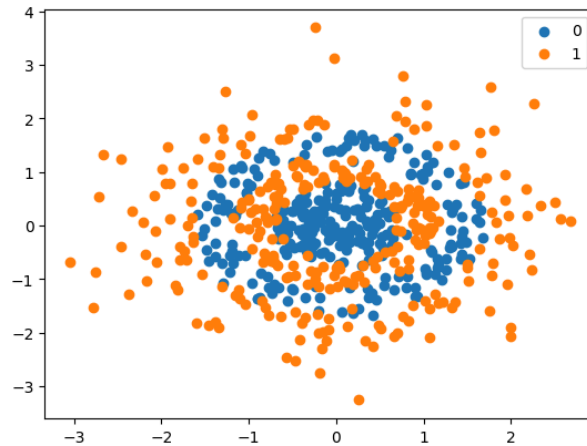


Figura 9: Scatter de puntos del apartado 4

2.d.a) Kernel lineal

Para ver como es la situación inicial del problema intentamos realizar la separación con un kernel lineal. Podemos observar, que incluso con la mejor combinación del CV, el f1 que obtenemos es del 43%.

```
params = {
    "C": (100, 1, 5, 10, 25, 50, 250),
    "kernel": ("linear",)
}
resultados_p4 = generic_svm_cv(dataset4, params)
resultados_p4

{'best': SVC(C=100, kernel='linear'),
 'params': {'C': 100, 'kernel': 'linear'},
 'scores': array([0.44585987, 0.37735849, 0.47191011]),
 'mean_score': 0.4317094918456844}
```

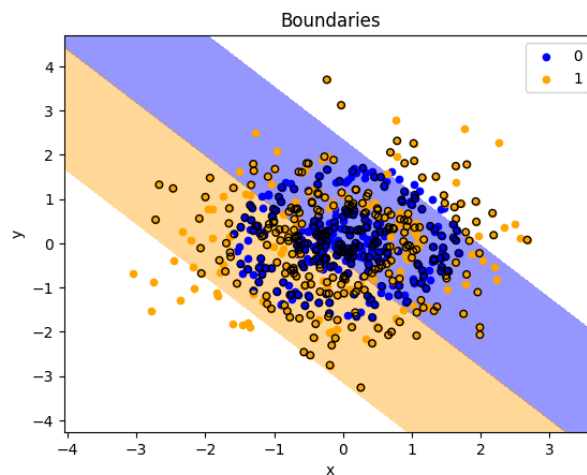


Figura 10: SVM 1.4

2.d.b) Kernel Polinómico

De manera similar al apartado anterior, intentamos realizar una aproximación con un kernel polinómico para ver si podemos tener una base mejor. En este caso, a través de un polinomio de grado 4 obtenemos un 60% de f1, que supera por un 17% lo que habíamos conseguido con el lineal.

```

params = {
    "C": (100,25,50,250),
    "kernel": ("poly",),
    "degree": (3,4,5,6),
}
resultados_p4 = generic_svm_cv(dataset4, params)
resultados_p4

{'best': SVC(C=100, degree=4, kernel='poly'),
 'params': {'C': 100, 'degree': 4, 'kernel': 'poly'},
 'scores': array([0.59354839, 0.58441558, 0.63803681]),
 'mean_score': 0.6053335937761032}

```

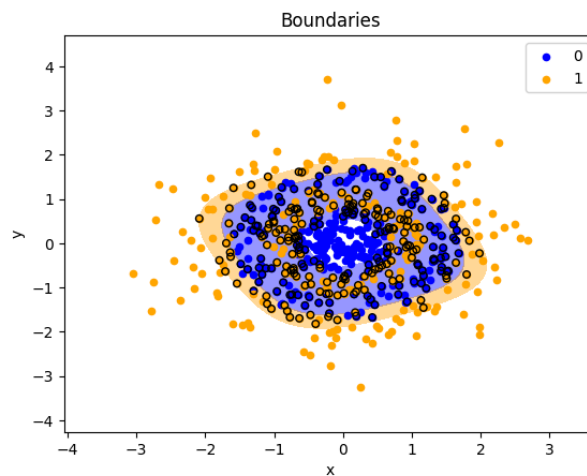


Figura 11: SVM 1.4

2.d.c) Kernel Radial

Por último, usamos el kernel que sabemos que debería aproximarse mejor a la forma. En este caso el kernel radial rbf supera por un 30% lo obtenido a través del polinómico, teniendo una media de f1 del 0.917%.

```

params = {
    "C": (100, 1,5,10,25,50,250,1000),
    "kernel": ("rbf",)
}
resultados_p4 = generic_svm_cv(dataset4, params)
resultados_p4

{'best': SVC(C=1000),
 'params': {'C': 1000, 'kernel': 'rbf'},
 'scores': array([0.91176471, 0.89447236, 0.94472362]),
 'mean_score': 0.9169868952606168}

```

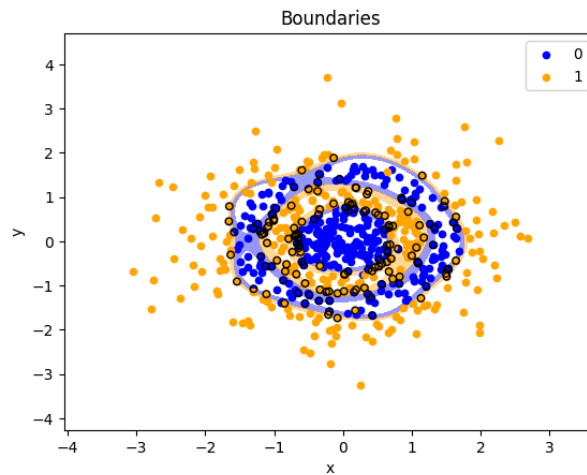


Figura 12: SVM 1.4

2.d.d) Comparación

Podemos observar la misma conclusión que sacábamos del apartado anterior, con la diferencia de que ahora la diferencia es mucho más apreciable.

- Como los kernels polinómico y lineal no tenían la base apropiada, incluso intentando encontrar su mejor variación los resultados son mediocres. Sin embargo, usando un kernel que se aplica bien al contexto (en este caso radial) podemos obtener resultados mucho mejores que con el resto de kernels.

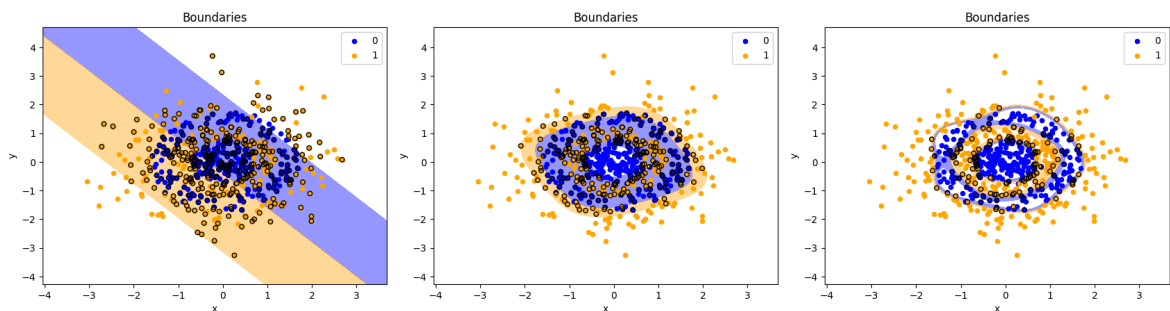


Figura 13: Comparación de los tres tipos de kernels probados.

2.d.e) ¿Como funciona el kernel radial?

El kernel radial o gaussiano es el que se aproxima en el ejemplo que se presenta al principio del informe. Para hacer la transformación se usa $(X^2 - Y^2)$. Esto significa que se transforman los puntos en pseudo circunferencias.

3) Caso Práctico

Ahora vamos a realizar la evaluación sobre el caso practica con el *dataset* de SONGS. A diferencia de los datasets, SONGS tiene 13 columnas (+ label) de datos por lo que su complejidad es bastante mayor.

3.a) Preprocessing

Para hacer el preprocesado de datos solamente vamos a realizar una normalización. Esto se debe a que pca/lda serán consideradas a la hora de realizar mejoras.

Los datos se normalizan porque para las operaciones del SVM facilitan mucho las cosas y hacen que se ejecute mucho más rápido.

A continuación un ejemplo de normalización:

```
def normalized_df(df, columns):
    mean = df[columns].mean()
    std = df[columns].std()

    return ((df[columns]-mean) / std)

dataset_song_norm = normalized_df(dataset_song, x_columns)
dataset_song_norm["y"] = dataset_song["y"]
dataset_song_norm
```

	acousticness	danceability	duration_ms	energy	instrumentalness	key	liveness	loudness	mode	speechiness	tempo	time_signature	valence	y
0	-0.682298	1.332543	-0.508725	-1.177408	-0.407763	-0.916219	-0.166250	-0.454418	0.795540	3.762151	1.066445	0.124017	-0.852827	1
1	0.043886	0.773638	0.983472	-1.534087	-0.465568	-1.190324	-0.346368	-0.881354	0.795540	-0.147493	1.441966	0.124017	0.368878	1
2	-0.589217	1.363594	-0.739179	-1.282034	-0.487079	-0.916219	-0.204846	-0.016582	0.795540	2.183171	-1.744734	0.124017	-1.309955	1
3	1.601643	-0.772668	-0.571995	-1.633957	1.379087	-0.093905	-0.634558	-2.166683	0.795540	-0.740166	-1.316638	0.124017	-1.079369	1
4	-0.029194	0.369984	1.788040	-0.573431	1.386409	-0.093905	1.596339	-1.212855	-1.256384	-0.258689	1.963633	0.124017	1.647219	1
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
2012	-0.717453	-0.213762	0.342732	1.190942	-0.478088	-1.190324	-0.397831	0.952931	0.795540	2.672432	-1.747282	0.124017	-1.156231	0
2013	-0.384208	1.711357	-0.782176	1.000713	-0.481822	-1.190324	-0.888010	1.175703	0.795540	0.426277	-0.433278	0.124017	1.497539	0
2014	-0.688567	0.115371	-0.477011	1.205209	-0.473329	-1.464429	0.148958	1.227808	0.795540	0.159407	1.067194	0.124017	-0.108477	0
2015	-0.715222	-0.381434	-0.740484	1.476285	1.990446	-1.190324	-0.640347	1.156563	0.795540	0.448517	1.064534	0.124017	0.510466	0
2016	-0.710722	-1.070751	-0.509701	1.110094	-0.487794	1.002514	0.174689	0.229850	0.795540	0.537473	2.563544	0.124017	-0.383563	0

2017 rows x 14 columns

Figura 14: Dataset Normalizado

3.b) Baseline

Para calcular el baseline de los datos, vamos a aplicar unicamente la normalización y vamos a ver que tipo de kernel funciona mejor para $C = 1$ y con el ganma por defecto y en auto (factor de penalización/aprendizaje).

```
params = {
    "C": (1,),
    "kernel": ('linear', 'poly', 'rbf', 'sigmoid'),
    "gamma": ('scale', 'auto'),
}
resultados_song = generic_simple_cv(
    dataset_song_norm,
    params,
    x_columns=x_columns,
    out_split=5
)
```

Podemos ver que a nivel empírico el Kernel que mejor funciona es el RBF, que nos proporciona un baseline de 72% de F1-score de media para los 5 *folds* de validación cruzada.

```
resultados_song

{'best': SVC(C=1),
 'params': {'C': 1, 'gamma': 'scale', 'kernel': 'rbf'},
 'scores': array([0.63709259, 0.64967599, 0.72191796, 0.55791597, 0.63709259,
                  0.64754354, 0.72089506, 0.55476022]),
 'best_score': 0.7219179601026989}
```

3.c) Reducción de dimensionalidad con PCA

Lo siguiente que probamos para el preprocesado de datos es realizar un PCA. Para ello primero probamos a mostrar gráficamente la cantidad de varianza explicada en base al número de dimensiones del PCA.

```
pca = PCA()  
_ = pca.fit(dataset_song_norm[x_columns])  
plot_variancia_acumulada(pca)
```

Como se puede ver en la Figura 15, hay muy poca covarianza de los atributos de los datos, pues son necesarias la mayor parte de las dimensiones para explicar el 90 o 95% de la varianza. Es por esta razón que finalmente decidimos no utilizar PCA, pues reducir los datos en sólo una o dos dimensiones no ayudaría demasiado al algoritmo a converger más rápido, y solo reduciría el *accuracy* del modelo innecesariamente.

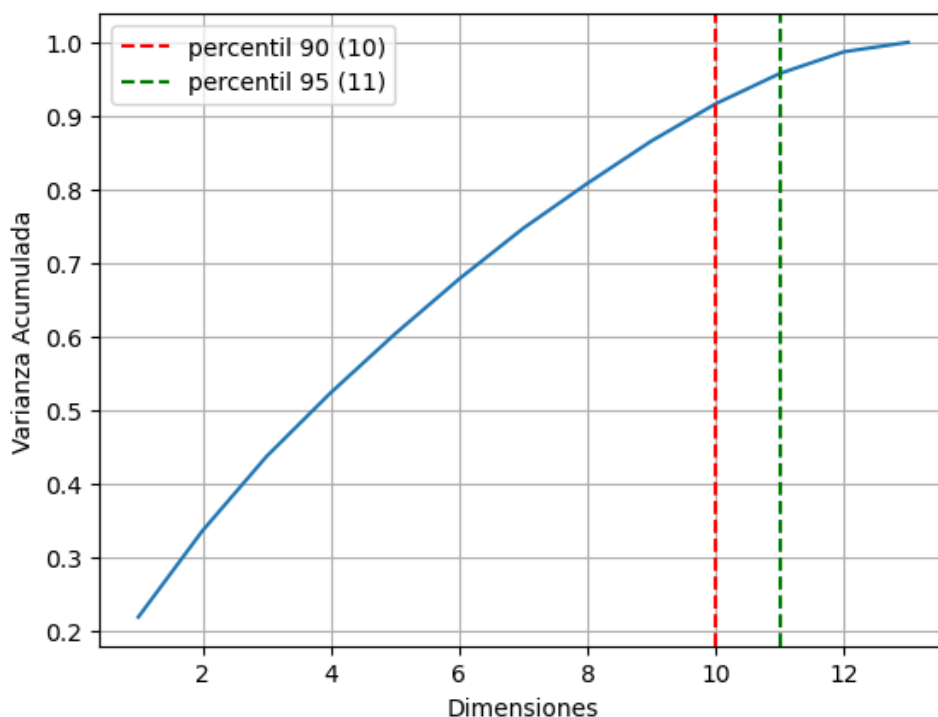


Figura 15: Varianza acumulada explicada para un número de dimensionades dada para el PCA

3.d) Búsqueda de parámetros

Para encontrar el mejor modelo posible decidimos buscar extensivamente la mayor cantidad de combinaciones de parámetros posibles para cada uno de los tipos de Kernel. Para agilizar la búsqueda, se utilizaron múltiples hilos aprovechando de esta capacidad de la implementación de validación cruzada de sklearn.

En primer lugar probamos el kernel lineal, que como solo tiene que probar el hiperparámetro C , se evalúa muy rápidamente. Debido a esto también nos pudimos permitir probar una gran cantidad de valores para C . En este caso, probamos todos los valores de 0 a 100.

```
params = {  
    "C": list(np.arange(0,100+1, step=1)),
```

```

    "kernel": ("linear",),
}
resultados_song = generic_simple_cv(
    dataset_song_norm,
    params,
    x_columns=x_columns,
    out_split=5,
    verbose = 2,
    n_jobs = 4,
)

```

Como se puede ver, el mejor resultado de F1-score medio en validación cruzada es del 63%, bastante malo respecto al *baseline*. Esto parece indicar que los datos no son realmente separables de forma lineal.

```

{'best': SVC(C=46, kernel='linear'),
 'params': {'C': 46, 'kernel': 'linear'},
 'scores': [...],
 'best_score': 0.6315685208362348}

```

Lo siguiente que se probó son los kernels polinomiales. Estas son las pruebas que tardaron más, pues no solo hay que probar gran cantidad de hiperparámetros (como el grado del polinomio, gamma, el coeficiente, etc), sino que, además, ciertas combinaciones de hiperparámetros tardan mucho tiempo en converger (desde decenas de segundos hasta minutos), especialmente cuando el coeficiente es relativamente alto.

```

params = {
    "C": list(np.arange(0,100+1, step=10)),
    "kernel": ("poly",),
    "degree": (3,4,5),
    "gamma": ("scale", 0.1, 0.2),
    "coef0": list(np.linspace(-10,0.5,55)),
}
resultados_song = generic_simple_cv(
    dataset_song_norm,
    params,
    x_columns=x_columns,
    out_split=5,
    verbose = 2,
    n_jobs = 4,
)

```

Todo ello nos consiguió un resultado bastante similar al *baseline* en *accuracy*, si bien se tardó bastante más en encontrarlo.

```

{'best': SVC(C=10, coef0=0.5, degree=4, kernel='poly'),
 'params': {'C': 10,
 'coef0': 0.5,
 'degree': 4,
 'gamma': 'scale',
 'kernel': 'poly'},
 'scores': [...],
 'best_score': 0.72336646685865}

```

Además, como el coeficiente de la mejor solución encontrada estaba justo en el límite del rango de coeficientes buscados (0.5), se decidió centrar más la búsqueda solo con polinomios de grado 4 en el rango de coeficientes de -1 a 1 . Se probó también una mayor cantidad de valores posibles de C .

Nótese que tuvimos que especificar un número máximo de iteraciones para converger, pues había ciertas combinaciones de parámetros que no parecían converger nunca, o, al menos, no convergieron en una cantidad razonable de tiempo (casi una hora).

```
params = {
    "C": list(np.arange(0,100+1, step=1)),
    "kernel": ("poly",),
    "degree": (4,),
    "gamma": ("scale",),
    "coef0": list(np.linspace(-1.0,1.0,20)),
    "max_iter": (10000000,),
}
resultados_song = generic_simple_cv(
    dataset_song_norm,
    params,
    x_columns=x_columns,
    out_split=5,
    verbose = 2,
    n_jobs = 4,
)
```

Todo ello consiguió mejorar muy ligeramente los resultados, dándonos la mejor solución con kernel polinomial encontrada hasta ahora.

```
{'best': SVC(C=8, coef0=0.7894736842105261, degree=4, kernel='poly',
max_iter=10000000),
'params': {'C': 8,
'coef0': 0.7894736842105261,
'degree': 4,
'gamma': 'scale',
'kernel': 'poly',
'max_iter': 10000000},
'scores': [...],
'best_score': 0.7299835774024117}
```

Seguidamente probamos el kernel sigmoideal. Este kernel no tenía que probar varios grados como el kernel polinomial, por lo que acabó bastante antes. También por ello se probaron muchos más valores posibles para gamma y el coeficiente.

```
params = {
    "C": list(np.arange(0,100+1, step=10)),
    "kernel": ("sigmoid",),
    "gamma": ("scale", 0.1, 0.2, 0.3, 0.4, 0.5),
    "coef0": list(np.linspace(-10,10, 100)),
}
resultados_song = generic_simple_cv(
    dataset_song_norm,
    params,
    x_columns=x_columns,
```

```

out_split=5,
verbose=2,
n_jobs=4,
)

```

Esto nos llevó a un F1-score de 69%, nuevamente, peor que el *baseline* que encontramos inicialmente.

```

{'best': SVC(C=100, coef0=-5.555555555555555, gamma=0.1, kernel='sigmoid'),
'params': {'C': 100,
'coef0': -5.555555555555555,
'gamma': 0.1,
'kernel': 'sigmoid'},
'scores': [...],
'best_score': 0.6973963219694614 }

```

Por último, probamos extensivamente el kernel RBF, que es el mismo kernel de nuestro baseline. Como hay muy pocos hiperparámetros que probar, se probaron extensivamente valores de C y de gamma. En el caso de gamma, se probaron valores en un rango muy concreto, de 0.001 a 0.1, porque empíricamente se comprobó que en varias ejecuciones de prueba que todas las buenas soluciones con kernels RBF tenían valores de gamma muy bajos.

```

params = {
    "C": list(np.linspace(40,80,100)),
    "kernel": ("rbf",),
    "gamma": ["scale"]+list(np.linspace(1/100,1/10,100)),
}
resultados_song = generic_simple_cv(
    dataset_song_norm,
    params,
    x_columns=x_columns,
    out_split=5,
    verbose = 2,
    n_jobs = 4,
)

```

Como se puede apreciar, la solución encontrada con $C = 63.43$ y gamma = 0.06 es la mejor solución encontrada hasta ahora, con un F1-score de 74,48%, por encima del baseline del 72,19%.

```

{'best': SVC(C=63.43434343434343, gamma=0.060000000000000005),
'params': {'C': 63.43434343434343,
'gamma': 0.060000000000000005,
'kernel': 'rbf'},
'scores': [...],
'best_score': 0.7448220920710862 }

```

Todos los resultados anteriores se muestran más en detalle en la Figura 16, donde se puede apreciar más fácilmente la diferencia de *accuracy* de los distintos modelos.

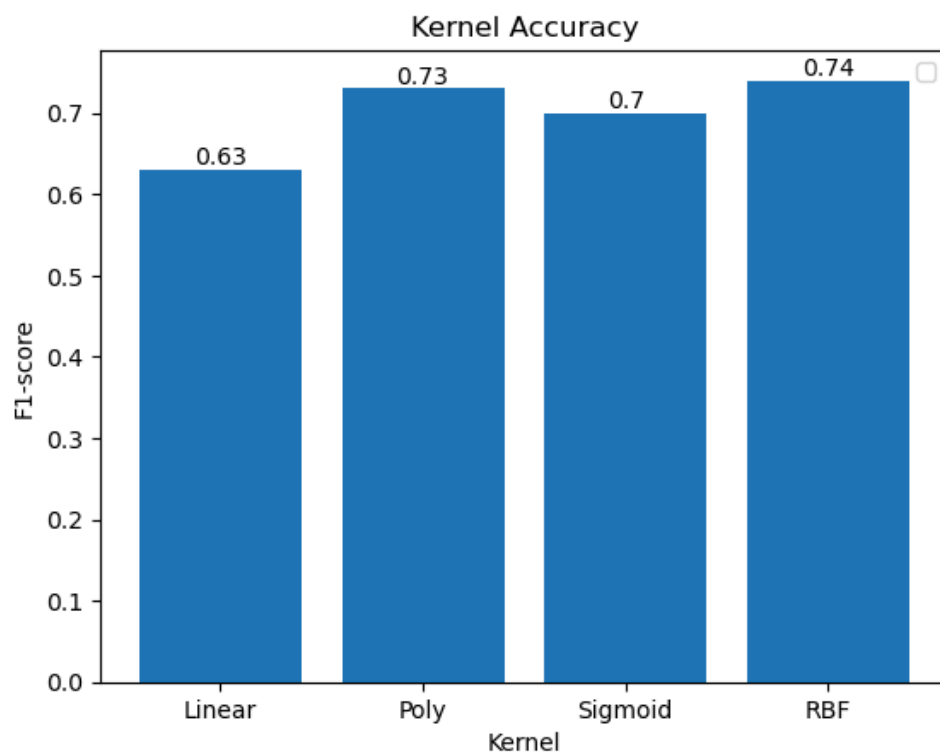


Figura 16: *Accuracy* de validación cruzada de los distintos Kernels probados en el *dataset*.